implement thread

> code analysis는 0x41. added code analysis에 작성해 두었습니다!

0x10. REQ; requirement

> 한마디로 process에서 thread로 확장해라

- task단위로서 PCB는 유지한다.
- 하지만 memory 관리정보는 독립적으로 관리한다.

0x20. DESIGN

0x21. thread로 바뀌는 큰 그림

1.

1. clone으로 생성되면 group leader thread와 page table 공유
2. user area
    1. code/text : shared mapping
    2. data : shared mapping
    3. heap : shared mapping
    4. user stack : thread별 다른 VA range → thread별 다른 physical page
    5. trapframe : thread별 다른 VA → thread별 다른 physical page
3. kernel area
    - kernel text/data/device mapping : 동일
    - kernel stack : thread/proc마다 다름
    - trampoline : 동일

2. 메모리 구조 - physical memory address

구간	주소	용도	누가 씀
Boot ROM	`0x00001000`	QEMU boot ROM	QEMU
CLINT	`0x02000000`	timer / inter-processor interrupt	kernel
PLIC	`0x0c000000`	external interrupt controller	kernel
UART0	`0x10000000`	console I/O	kernel
Virtio disk	`0x10001000`	disk I/O	kernel
Kernel load base	`0x80000000`	kernel entry/text/data 시작	kernel
Free physical pages	`end ~ PHYSTOP`	allocator가 user page, kernel page에 배분	kernel allocator
RAM end	`PHYSTOP = 0x80000000 + 128MB`	xv6가 쓰는 physical RAM 끝	kernel

3. 메모리 구조 - kernel VA

높은 주소 → 낮은 주소	symbol / macro	크기	의미	특징
최상단 근처	`TRAMPOLINE = MAXVA - PGSIZE`	1 page	user/kernel 전환 code	모든 process/thread가 같은 page를 mapping
그 아래	`KSTACK(p)`	proc마다 1 page + guard page	kernel mode stack	thread마다 별도
kernel base	`KERNBASE = 0x80000000`	kernel 크기만큼	kernel text/data	공유
allocator 영역	`end ~ PHYSTOP`	남은 RAM	physical page allocator	user page, trapframe, pagetable 등에 배분
device MMIO	`UART0`, `VIRTIO0`, `PLIC` 등	device별	device register mapping	kernel만 접근

4. 메모리구조 - user VA

> 낮 -> 높

구간	주소 범위	mapping 대상	공유 여부	설명
text	`0x0` 근처부터	program code page	thread 간 공유	실행 code
data / bss	text 뒤	global/static data	thread 간 공유	global variable
initial user stack	data 뒤	user stack page	보통 leader의 초기 stack	xv6 기본 process stack
heap	stack 뒤 ~ `USERTOP` 아래	malloc/sbrk 영역	thread 간 공유	`CLONE_VM` thread들이 같이 봄
reserved user memory cap	`USERTOP = TRAPFRAME - NTHREAD * PGSIZE`	user heap/stack이 넘으면 안 되는 top	shared mm의 limit	trapframe slot 보호용 경계
trapframe slots	`USERTOP ~ TRAPFRAME`	thread별 trapframe physical page	VA는 thread별 다름	`NTHREAD`개 slot
leader trapframe	`TRAPFRAME = TRAMPOLINE - PGSIZE`	group leader trapframe	leader 전용	slot 0처럼 취급 가능
trampoline	`TRAMPOLINE = MAXVA - PGSIZE`	trampoline code	전체 공유	user/kernel 전환

0x22. Extend memory management

> to support shared address spaces

trapframe VA도 thread별로 구분

1. trampoline이 그 thread의 trapframe VA를 사용

2. trapframe을 별도로 두기

mm refcount로 shared address space 생명주기를 관리

1. `find_free_tf_va`

> WHY(의도/목적): 사용되지 않은 VA 위치를 제공

2. `mm_get()`

> WHY(의도/목적): 하나의 address space를 새 task가 공유하기 시작할 때 참조 수를 올리는 함수

- ref_count++; (a new task starts sharing)

3. `mm_put()`

> WHY(의도/목적): mm_struct에 대한 참조를 하나 내려놓는 함수

- ref_count--; (a task lets go)
- mm_put(mm): **공유 address space 자체의 lifetime** 정리
  - 예: refcount--, 마지막이면 page table과 user memory 해제

4. freeproc()

- per-thread trapframe slot unmap inside
- freeproc(p): **이 proc 전용 자원** 정리
  예: p->trapframe, p->tf_va 매핑, file/cwd 등

0x23. threading primitives

1. clone system call 분석

```
int clone(void (*fn)(void *), void *arg, void *stack,
          int n_pages, int flags);
```

- fn: 새 thread가 가장 먼저 실행할 함수
- arg: fn에 넘겨줄 arguments
- stack: 새 thread가 쓸 user stack의 시작 주소
- n_pages: 몇 페이지 줄지
- flags: 프로세스처럼 만들지, thread처럼 만들지 정하는 플래그

2. `thread_create`

1) user stack을 어떻게 마련할 것인가?

1. IDEA1. malloc으로 heap을 사용.
2. IDEA2. mmap 스타일 system call을 별도로 정의해 스택 할당
3. IDEA3. user address space 위쪽에 thread stack 영역을 예약

> -> BUT.. 바꿀 수 있는 파일은 3가지 뿐이므로 2,3는 기각
> -> malloc으로 heap에 할당하되, 사용은 stack처럼.

3. join 설계

> 의도/목적: thread group 안에서 종료된 sibling thread를 기다렸다가 회수하는 커널 내부 함수

- join caller = group leader (이걸로 고정함!!!)
- join target = group_leader가 자신인 ZOMBIE thread들

함수	회수 대상	관계	**address space**
wait()	child process의 ZOMBIE	parent-child	별도 mm
join()	sibling thread의 ZOMBIE	같은 thread group	같은 mm 공유

1) sibling thread를 어떻게 식별할 것인가?

- 조건
  1. group_leader동일 / 같은 tgid
  2. proc->state == ZOMBIE
- NPROC이 제한되어 있기에, 아쉽지만 그냥 선형 조회하는 식으로 처리.

2) 어떤 자원을 회수해야 하는가?

- trapframe VA, PA
- mm_struct

3) stack_addr에 할당

- user stack부분에 대한 주소를 줘야 함.
- 이때 다음 함수 사용 `copyout()`
  - 커널 변수를 유저 주소에 써주는 함수

4. thread_join

> free stack returned by join() system call using free()

- thread_clone에서 할당된 stack을 free

0x24. wait

purpose

1. 내 child process 중 ZOMBIE 찾기
2. 없으면 sleep
3. 찾으면 exit status 반환
4. process 자원 정리

0x25. kill

purpose

1. tgid == pid인 모든 thread에 killed 표시

2. sleeping thread 깨우기
3. 각 thread가 kernel/user 경계에서 exit(-1)로 빠지게 만들기

0x26. exit

purpose

1. non-leader thread가 exit()
   - -> 그 thread만 종료
   - -> ZOMBIE
   - -> join 대상
2. group leader가 exit()
   - -> process 전체 종료
   - -> sibling threads도 kill
   - -> child process reparent
   - -> parent wait 대상

freeproc을 하면 안되는 이유?

freeproc(p)는 p->trapframe, p->kstack, pagetable reference, proc slot 등을 정리함
그런데 exit 중인 p는 아직 p->lock, p->context, kstack을 사용해서 sched() 해야 함

0x27. exec

purpose

- exec 이후에도 유지되는 것은
  - pid, 열린 파일, cwd, 커널 proc 구조체
- 사라지는 것은 **기존 코드/데이터/힙/스택 같은 사용자 주소 공간**

thread확장시 고려사항

- **thread들은 주소 공간을 공유하므로, exec가 주소 공간을 바꾸면 같은 group의 다른 thread들이 즉시 깨진다.**
- 다른 thread가 기존 코드에서 실행 중이거나 기존 user stack을 쓰고 있는데 leader가 page table을 통째로 교체하면, 그 thread의 epc, sp, user stack, trapframe mapping 관계가 모두 무효가 됨.

설계

- **non-leader thread는 exec 불가** : sibling thread가 자기 마음대로 전체 process image를 바꾸면 group 전체 의미가 꼬이므로 -1.
- **leader만 exec 가능**
- **exec 전에 sibling thread를 모두 죽이고 회수해야 함**
- **그 후 단일 thread 상태에서 새 주소 공간으로 교체**

0x30. IMPL

0x31. vm.c의 api 분석

1. kernel page table 관리

- kvmmake(): 커널용 page table을 새로 만들고 UART, virtio disk, PLIC, kernel text/data, trampoline, kernel stack들을 매핑
  - 사용 위치: kvminit()에서 부름. 부팅 초기에 딱 한 번 커널 page table을 만들 때 씀.
- kvmmap(): 커널 page table에 VA -> PA 매핑을 추가하는 wrapper입니다. 내부에서 mappages()를 부름
  - 사용 위치: kvmmake(), proc_mapstacks().
- kvminit(): 전역 kernel_pagetable을 초기화
  - 사용 위치: kernel/main.c (line 20), boot CPU가 커널 page table 만들 때.
- kvminithart() : 현재 CPU의 satp 레지스터에 kernel_pagetable을 넣어서 paging을 켬
  - 사용 위치: kernel/main.c (line 21), 각 CPU가 paging 활성화할 때.

2. page table 탐색/매핑 기본 도구

- walk(pagetable, va, alloc)
  - 3-level page table을 따라 내려가서 va에 해당하는 leaf PTE 주소를 반환. alloc=1이면 중간 page table page가 없을 때 kalloc()으로 만듦
  - 사용 위치: 거의 모든 VM 함
    수. mappages, walkaddr, uvmunmap, uvmcopy, uvmclear, copyout, ismapped.
- walkaddr(pagetable, va): 유저 가상주소 va가 실제 어느 물리주소에 매핑되는지 반환환. PTE_V, PTE_U가 있어야 성공
  - 사용 위치: copyin, copyout, copyinstr, exec.c의 loadseg().
- mappages(pagetable, va, size, pa, perm) 특정 VA 범위를 PA 범위에 매핑. 필요하면 walk(..., alloc=1)로 page table page도 만듦
  - 사용 위치: kvmmap, proc_pagetable, uvmalloc, uvmcopy, vmfault.

3. user address space 관리

- uvmcreate() : 빈 유저 page table root page 하나를 만듦
  - 사용 위치: proc_pagetable()에서 새 프로세스 page table 만들 때.
- uvmalloc(pagetable, oldsz, newsz, xperm): 유저 주소공간을 oldsz에서 newsz까지 키움. 각 페이지마다 kalloc()으로 물리 페이지를 할당하고 mappages()로 매핑
  - 사용 위치: exec()가 프로그램 segment와 stack을 만들 때, growproc()가 eager sbrk() 처리할 때.
- uvmdealloc(pagetable, oldsz, newsz) : 유저 주소공간을 줄임. 줄어든 범위의 page들을 uvmunmap(..., do_free=1)로 해제
  - 사용 위치: growproc()에서 sbrk(-n) 처리, uvmalloc() 실패 롤백.
- uvmunmap(pagetable, va, npages, do_free) : VA 범위의 PTE를 지움. do_free=1이면 매핑된 물리 페이지도 kfree()
  - 사용 위치: proc_freepagetable, uvmdealloc, uvmfree, uvmcopy 실패 처리,
    trampoline/trapframe 매핑 해제.
- freewalk(pagetable): page table page들만 재귀적으로 해제. 단, leaf mapping이 남아 있으면 panic
  - 즉 물리 데이터 페이지는 먼저 uvmunmap(..., do_free=1)으로 해제되어 있어야 함.
- uvmfree(pagetable, sz) : 유저 데이터 페이지들을 먼저 해제하고, 그 다음 page table page들을 freewalk()로 해제

- 사용 위치: proc_freepagetable(), proc_pagetable() 실패 처리 등.

4. fork/exec 관련

- uvmcopy(old, new, sz) : fork() 때 부모 유저 메모리를 자식 page table로 복사. 각 매핑된 페이지마다 새 물리 페이지를 kalloc()하고 내용을 memmove()로 복사.
  - 사용 위치: kclone()의 fork 경로.
- uvmclear(pagetable, va) : 특정 PTE에서 PTE_U를 제거해서 유저 모드 접근을 막음
  - 사용 위치: exec()에서 stack guard page 만들 때. guard page는 매핑은 되어 있지만 유저가 접근하면 fault가 나야 하니까

5. kernel ↔ user 메모리 복사

- copyout(pagetable, dstva, src, len) : 커널 메모리 src에서 유저 주소 dstva로 복사
  - 예: wait(&status)에서 status를 유저 포인터에 써줄 때.
  - 사용 위치: file.c, pipe.c, sysfile.c, proc.c, exec.c.
- copyin(pagetable, dst, srcva, len) : 유저 주소 srcva에서 커널 버퍼 dst로 복사.
  - 예: syscall 인자로 받은 유저 버퍼 읽기.
  - 사용 위치: syscall.c, pipe.c, proc.c.
- copyinstr(pagetable, dst, srcva, max) : 유저 주소에서 null-terminated string을 커널로 복사
  - 사용 위치: syscall.c의 syscall string argument 처리.
  - 예: exec(path, argv)의 path, open(path) 같은 문자열.

6. lazy allocation

- vmfault(pagetable, va, read)
  - lazy sbrk()로 mm->sz만 늘려둔 상태에서 실제 접근이 발생하면 물리 페이지를 할당하고 매핑
  - 사용 위치: trap.c의 page fault 처리, 그리고 copyin/copyout이 lazy page를 만났을 때.
- ismapped(pagetable, va) : 해당 VA에 valid PTE가 있는지 확인
  - 사용 위치: vmfault() 내부

0x32. `kclone`

```
int
kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, int flags)
{
  int i, pid;
  struct proc *np;
  struct proc *p = myproc();

  if ((np = allocproc()) == 0)
    return -1;

  if (flags & CLONE_VM) { // === thread path ===
    // mm_struct는 공유
    struct mm_struct *m = p->mm;
    if(mm_get(m) < 0)
      goto fail;
    np->mm = m;
```

```
    // trapframe은 구분
    uint64 tf_va = find_free_tf_va(m->pagetable);
    if(tf_va == 0)
      goto fail;
    np->tf_va = tf_va;
    if (mappages(m->pagetable, tf_va, PGSIZE, (uint64)np->trapframe, PTE_R |
PTE_W ) == -1) {
      goto fail;
    };
    *(np->trapframe) = *(p->trapframe);

    // thread group 설정
    np->group_leader = p->group_leader;
    np->tgid = p->tgid;
    np->ustack = (void*)stack;

    np->trapframe->epc = fn;
    np->trapframe->a0 = arg;
    np->trapframe->sp = stack + n_pages * PGSIZE; // 초기 stack pointer이므로
  }

// ---------------------------------------------
```

1. clone vs fork

- CLONE_VM이 있으면 **thread** 생성
- CLONE_VM이 없으면 **process fork**

2. 새 proc을 만들지만 address space은 복사하지 않는다

```
struct mm_struct *m = p->mm;
mm_get(m);
np->mm = m;
```

참고

```
int
mm_get(struct mm_struct *mm)
{
  acquire(&mm->lock);

  if(mm->refcount < 1)
    panic("mm_get: bad refcount");

  if(mm->refcount >= NTHREAD){
    release(&mm->lock);
    return -1;
  }

  mm->refcount++;
```

```
  release(&mm->lock);
  return 0;
}
```

3. 대신 trapframe은 공유하지 않는다

```
uint64 tf_va = find_free_tf_va(m->pagetable);
mappages(m->pagetable, tf_va, PGSIZE, (uint64)np->trapframe, ...);
*(np->trapframe) = *(p->trapframe);
```

참고

```
uint64
find_free_tf_va(pagetable_t pagetable)
{
  for (int i = 0; i < NTHREAD; i++) {
    uint64 loc = TRAPFRAME - i * PGSIZE;
    if (ismapped(pagetable, loc)) continue;
    return loc;
  }
  return 0;
}
```

4. 부모의 next instruction이 아닌 유저가 넘긴 함수부터 실행

```
np->trapframe->epc = fn;
np->trapframe->a0 = arg;
np->trapframe->sp = stack + n_pages * PGSIZE;
```

5. thread group 설정

```
np->group_leader = p->group_leader;
np->tgid = p->tgid;
np->ustack = (void*)stack;
```

0x33. `freeproc`

```
static void
freeproc(struct proc *p)
{
  struct mm_struct *m = p->mm;

  if(p->trapframe) {
    int freed_by_unmap = 0;

    if(m && m->pagetable && p->tf_va) {
      acquire(&m->lock);
      if(ismapped(m->pagetable, p->tf_va)) {
        uvmunmap(m->pagetable, p->tf_va, 1, 1); // VA(trapframe) + PA 정리
```

```
        freed_by_unmap = 1;
      }
      release(&m->lock);
    }

    if(!freed_by_unmap)
      kfree(p->trapframe);

    p->trapframe = 0;
  }

  p->tf_va = 0;

  if(m) {
    mm_put(m);
    p->mm = 0;
  }

  // PCB정리
  cleantask(p);
}
```

1. trapframe을 반드시 회수한다

1) 이때 w/ flag을 활용하여 다음 케이스를 방지.

> allocproc()은 trapframe을 먼저 kalloc()한다. 그런데 kclone() 도중 page table에 trapframe을 mapping 하기 전에 실패할 수 있다

- 따라서 uvmunmap을 하는 경우, 단순히 kfree를 하는 경우를 분리한다.

2) address space을 무조건 free하지 않는다. mm_put()으로 "참조 하나를 반납"할 뿐이다

```
void
mm_put(struct mm_struct *mm)
{
  acquire(&mm->lock);
  mm->refcount--;
  int ref = mm->refcount;
  release(&mm->lock);

  if (ref == 0) {
    if(mm->pagetable)
      proc_freepagetable(mm->pagetable, mm->sz);
    kfree(mm);
  }
}
```

2. proc table entry를 재사용 가능한 상태로 만든다

```
static void
cleantask(struct proc *p)
```

```
{
  p->pid = 0;
  p->parent = 0;
  p->name[0] = 0;
  p->chan = 0;
  p->killed = 0;
  p->xstate = 0;
  p->state = UNUSED;
  p->group_leader = 0;
  p->tgid = 0;
  p->ustack = 0;
}
```

0x34. kjoin

```
int
kjoin(uint64 stack_addr)
{
  struct proc *p;
  struct proc *cur_p = myproc();
  int found = 0;

  if(cur_p != cur_p->group_leader)
    return -1;

  acquire(&wait_lock);

  for (;;) {
    found = 0;
    for(p = proc; p < &proc[NPROC]; p++) {
      if (cur_p == p) {
        continue;
      }
      if (p->tgid != cur_p->tgid || p->group_leader == p) {
        continue;
      }

      // find sibling thread
      found = 1;
      acquire(&p->lock);

      if (p->state != ZOMBIE) {
        release(&p->lock);
        continue;
      }

      // 좀비 발견!
      int tid = p->pid;
      uint64 ustack = (uint64)p->ustack;

      if(stack_addr != 0 &&  // 스택 넘기기
```

```
          copyout(cur_p->mm->pagetable, stack_addr, (char *)&ustack,
sizeof(ustack)) < 0) {
        release(&p->lock);
        release(&wait_lock);
        return -1;
      }

      // trapframe회수 및 mm_struct도 회수
      freeproc(p);

      release(&p->lock);
      release(&wait_lock);
      return tid;
    }

    if (found == 0 || killed(cur_p)) {
      release(&wait_lock);
      return -1;
    }

    sleep(cur_p, &wait_lock);
  }

}
```

1. leader만 호출 가능; 같은 tgid를 가진 non-leader thread만 대상

2. wait_lock잡기

> "thread가 죽어서 ZOMBIE가 되는 순간”과 “leader가 그 thread를 찾고 잠드는 순간” 사이의 race를 막기 위해서

```
acquire(&wait_lock);

for (;;) {
  // proc table에서 join 대상 thread 찾기

  if (zombie thread found)
    freeproc(thread);

  if (found == 0 || killed(cur_p))
    return -1;

  sleep(cur_p, &wait_lock);
}
```

0x35. user level wrapper — thread create/join

```
int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
  void *st = malloc(n_pages * PGSIZE);
```

```
  int tid = clone(fn, arg, st, n_pages, CLONE_VM);
  if (tid < 0) {
    free(st);
    return -1;
  }

  return tid;
}

int
thread_join(void)
{
  void *stack;
  int tid = join(&stack);

  if (tid < 0) return -1;

  free(stack);
  return tid;
}
```

- 같은 주소 공간을 공유하는 thread를 만들기
- join()은 syscall.
  - kernel이 종료된 thread의 stack 주소를 user 변수 stack에 copyout()으로 써줌.
  - syscall이 return해서 user mode로 돌아온 뒤에야 free(stack)이 user mode에서 실행

0x36. `kexit`

```
void
kexit(int status)
{
  struct proc *p = myproc();

  if(p == initproc)
    panic("init exiting");

  // Close all open files.
  for(int fd = 0; fd < NOFILE; fd++){
    if(p->ofile[fd]){
      struct file *f = p->ofile[fd];
      fileclose(f);
      p->ofile[fd] = 0;
    }
  }

  begin_op();
  iput(p->cwd);
  end_op();

  p->cwd = 0;
```

```
  kdrainthreads(p);

  acquire(&wait_lock);
  // Give any children to init.
  reparent(p);

  // Parent might be sleeping in wait().
  // And the group leader might be sleeping in join().
  wakeup(p->parent);

  if (p->group_leader != p) {
    wakeup(p->group_leader);
  }

  acquire(&p->lock);
  p->xstate = status;
  p->state = ZOMBIE;

  release(&wait_lock);

  // Jump into the scheduler, never to return.
  sched();
  panic("zombie exit");
}
```

1. 일반 thread가 exit()하면 그 thread만 ZOMBIE가 됨

2. group leader가 exit()하면 sibling thread들을 먼저 죽이고 회수함

> kdrainthreads(p)를 호출해서 leader와 같은 thread group의 나머지 task들을 killed=1로 만들고, sleeping이면 깨운 뒤, 전부 ZOMBIE가 되어 회수될 때까지 wait.

```
void
kdrainthreads(struct proc *p)
{
  acquire(&wait_lock);

  struct proc *t;
  // 1. sibling까지 다 죽임(kill)
  if(p->group_leader == p) {

    for (t = proc; t < &proc[NPROC]; t++) {
      if (t == p) {
        continue;
      }
      if (t->group_leader == p) {
        acquire(&t->lock);
        t->killed = 1;
        if (t->state == SLEEPING) {
          t->state = RUNNABLE;
        }
        release(&t->lock);
      }
```

```
    }
    // 2. reaping
    for(;;){
      int have = 0; // scan flag

      for(t = proc; t < &proc[NPROC]; t++){
        if(t != p && t->group_leader == p){
          acquire(&t->lock);
          if(t->state == ZOMBIE){
            freeproc(t);
          } else {
            have = 1;
          }
          release(&t->lock);
        }
      }
      if(have == 0)
        break;
      sleep(p, &wait_lock);
    }
  }

  release(&wait_lock);
}
```

0x37. `kkill`

```
int
kkill(int pid)
{
  struct proc *p;
  struct proc *p_gl = 0;

  // pid통해 task특정
  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->pid == pid){
      p_gl = p->group_leader;
      if(p->state == SLEEPING){
        // Wake process from sleep().
        p->state = RUNNABLE;
      }
    }
    release(&p->lock);
  }

  if (p_gl == 0) {
    return -1;
  }

  int killed_any = 0;
  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
```

```
    if(p->group_leader == p_gl){
      p->killed = 1;
      killed_any = 1;
      if(p->state == SLEEPING){
        p->state = RUNNABLE;
      }
    }
    release(&p->lock);
  }
  return killed_any ? 0 : -1;
}
```

flow

1. 첫 번째 scan에서 pid에 해당하는 task를 찾음
2. 그 task의 group_leader를 저장
3. 두 번째 scan에서 같은 group_leader를 가진 모든 task에 killed = 1
4. sleeping task는 RUNNABLE로 깨움
5. 각 task는 이후 trap/syscall 복귀 경로에서 killed(p)를 보고 kexit(-1)로 종료됨

0x38. kexec

현재 process의 주소공간을 새 ELF 이미지로 갈아끼우되, thread group 모델에서는 leader만 exec 가능하게 한다

```
if (p->group_leader != p) {
  return -1;
}

beginop();

// Open the executable file.
...
// Read the ELF header.
...
// Is this really an ELF file?
...
// Load program into memory.
...

iunlockput(ip);
end_op();
ip = 0;

p = myproc();
uint64 oldsz = p->mm->sz;

// Allocate some pages at the next page boundary.
// Make the first inaccessible as a stack guard.
// Use the rest as the user stack.
...
// Copy argument strings into new stack, remember their addresses in ustack[].
```

```
...
// push a copy of ustack[], the array of argv[] pointers.
...
// a0 and a1 contain arguments to user main(argc, argv)
// argc is returned via the system call return value, which goes in a0.
...
// Save program name for debugging.
...
kdrainthreads(p);

// Commit to the user image.
acquire(&p->mm->lock);
oldpagetable = p->mm->pagetable;
p->mm->pagetable = pagetable;
p->mm->sz = sz;
release(&p->mm->lock);
p->trapframe->epc = elf.entry; // initial program counter = ulib.c:start()
p->trapframe->sp = sp; // initial stack pointer
proc_freepagetable(oldpagetable, oldsz);
return argc; // this ends up in a0, the first argument to main(argc, argv)
```

- exec는 주소공간 전체를 교체하는 syscall인데, 이 구현에서는 같은 thread group이 mm_struct와 pagetable을 공유함
- 어떤 thread 하나가 exec로 pagetable을 바꿔버리면, 다른 thread들은 기존 코드/스택/trapframe을 기준으로 실행 중일 수 있어서 깨짐

kdrainthreads의 위치

- 파일 열기/ELF 검증/새 pagetable 구성/stack 및 argv 복사까지 성공한 뒤에 thread들을 정리

```
void
kdrainthreads(struct proc *p)
{
  acquire(&wait_lock);

  struct proc *t;
  // 1. sibling까지 다 죽임(kill)
  if(p->group_leader == p) {

    for (t = proc; t < &proc[NPROC]; t++) {
      if (t == p) {
        continue;
      }
      if (t->group_leader == p) {
        acquire(&t->lock);
        t->killed = 1;
        if (t->state == SLEEPING) {
          t->state = RUNNABLE;
        }
        release(&t->lock);
      }
    }
    // 2. reaping
```

```
    for(;;){
      int have = 0; // scan flag

      for(t = proc; t < &proc[NPROC]; t++){
        if(t != p && t->group_leader == p){
          acquire(&t->lock);
          if(t->state == ZOMBIE){
            freeproc(t);
          } else {
            have = 1;
          }
          release(&t->lock);
        }
      }
      if(have == 0)
        break;
      sleep(p, &wait_lock);
    }
  }

  release(&wait_lock);
}
```

0x40. PS

> w/ 궁금증 해결

0x41. added code analysis

> linux like refactoring by 추가된 코드의 의도가 무엇인지 분석한다

1. thread group 개념 도입

```
// in proc.h
struct proc {
    // ..
    struct proc *group_leader; // group leader (self if not a thread)
    int tgid; // thread group id (== leader->pid)
    uint64 tf_va; // user VA where p->trapframe is mapped
    void *ustack; // user stack base (only set for threads)
}
```

- group_leader: thread 대표 정의
- tgid: 각 thread는 각자의 pid를 가지지만, 공통된 tgid를 공유한다.
- tf_va : thread는 같은 page table을 가지므로 trapframe을 공유하면 충돌한다. 따라서 각각 별도의 page table을 정의해야 한다.
- ustack: user stack 시작 포인터를 가리키게 하기 위함.

2. mm_struct 분리

```
struct mm_struct {
    pagetable_t pagetable; // page table root
    uint64 sz; // user memory size (bytes)
    int refcount; // number of tasks sharing this mm
    struct spinlock lock;
};
```

- 의도1: fork시
  - 별도 mm_struct생성
  - 즉, 별도 메모리(page table) 새로 생성
- 의도2: thread_create시 (clone)
  - page table 공유
  - clone될 때마다 refcount증가 + join시에 감소
  - refcount == 0이면 page table과 user memory 해제

3. trampoline/trap 경로가 sscratch 기반 per-thread trapframe으로 변경

- implement system call에서 했듯이, user mode에서 trap발생시 항상 stvec에 cpu가 trap발생시 이동할 주소를 저장한다.
- prepare_return은 uservec을 저장해둔다.

```
// in trap.c
void
prepare_return(void) {
    // ..
    w_stvec(trampoline_uservec);
    // ..
    w_sscratch(p->tf_va);
    // ..
}

// in trampoline.S
uservec:
    csrrw a0, sscratch, a0
    // 이후 save register to trapframe
```

1. 현재 thread의 trapframe va를 sscratch로 가져오기
2. a0와 sscratch(tf_va) swap
3. a0(tf_va)에 현재 cpu register 정보 저장

4. per-thread trapframe을 위한 address space 예약

```
TRAMPOLINE
TRAPFRAME                        slot 0, group leader
TRAPFRAME - 1*PGSIZE             slot 1
TRAPFRAME - 2*PGSIZE             slot 2
...
TRAPFRAME - (NTHREAD-1)*PGSIZE   slot 7
USERTOP 아래                     일반 user memory
```

- 이런구조로 USERTOP을 기준으로 그 위에 대한 접근을 막으면 됨.

5. fork syscall 제거, clone syscall 도입

> process 생성과 thread생성을 하나의 primitive system call을 정의함으로써 담당

6. join syscall 추가

	join	wait
목적/의도	thread group 내부의 thread회수	process 회수

0x42. why does forktest fail?

1. 에러 메시지의 의미.

> FAILED -- lost some free pages 31042 (out of 32445)

- 이전 free page가 32445, 이후가 31042
- 1xxx개의 Page가 leaking

2. 왜 leaking이 됐는가?

- forktest에서 fork()는 결국 kernel/proc.c (line 311)의 kclone(..., flags=0) 경로를 탐.
- 자식마다 mm_alloc()으로 새 mm_struct 페이지를 만들고, proc_pagetable()로 새 pagetable을 만들고, uvmcopy()로 부모 유저 메모리를 복사함.
- BUT kwait로 exit(0)인 자식 스레드를 다 정리하려고 하나 trapframe만 kfree()되고, 자식의 mm_struct, pagetable page들, uvmcopy()로 복사한 유저 물리 페이지들이 전부 반환되지 않음

0x43. PS; freeproc, mm_put 책임 경계 혼동

1. log

```
$ usertests forktest
usertests starting
test forktest: usertrap(): unexpected scause 0xc pid=4
            sepc=0x101010101010101 stval=0x101010101010101
usertrap(): unexpected scause 0xc pid=1
            sepc=0x101010101010101 stval=0x101010101010101
usertrap(): unexpected panscaicu:se 0xc  pid=3
            sepci=0nit exixti1ng
01
```

- trapframe 안의 epc가 0x010101...가 됐다
- trapframe 페이지가 kfree()된 뒤에도 그 trapframe을 다시 사용했다

2. problem code

```
static void
freeproc(struct proc *p)
{
```

```
  if (p->trapframe) {
    kfree((void *)p->trapframe);
    p->trapframe = 0;
  }
  p->tf_va = 0;

  if (p->mm) {
    mm_put(p->mm);
    p->mm = 0;
  }

  p->pid = 0;
  p->parent = 0;
  p->name[0] = 0;
  p->chan = 0;
  p->killed = 0;
  p->xstate = 0;
  p->state = UNUSED;
  p->group_leader = 0;
  p->tgid = 0;
  p->ustack = 0;
}

void
mm_put(struct mm_struct *mm)
{
    struct proc *p = myproc();
    acquire(&mm->lock);
    mm->refcount--;
    uvmunmap(mm->pagetable, p->tf_va, 1, 1);
    kfree(p->trapframe); // trapframe free 책임이 중복
    release(&mm->lock);
}
```

1. wait()에서 자식 pp를 정리할 때 흐름은 `freeproc(pp);`
    1. 위에 처럼 myproc호출시
    2. 이 순간 myproc()는 **자식 pp가 아니라 부모 프로세스**
2. 자식 정리 중 부모 trapframe을 kfree()
3. 부모가 user로 돌아가려 함
4. trapframe->epc가 0x0101010101010101
5. instruction page fault

0x43. 도대체 kernel stack은 thread에서 어떻게 관리되는 거지?

ㄱ. 부팅 중 **kernel page table** 만들 때

- kernel/vm.c `proc_mapstacks(kpgtbl);` 를 호출함.

ㄴ. **proc_mapstacks()**가 모든 **proc** 슬롯마다 **kernel stack** 페이지를 하나씩 할당함

```
// kernel/proc.c
```

```
for(p = proc; p < &proc[NPROC]; p++) {
    char *pa = kalloc();
    uint64 va = KSTACK((int) (p - proc));
    kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
}
```

- 즉 proc[0], proc[1], proc[2] ... 각각에 대해: 물리 페이지 하나 kalloc() 커널 가상주소 KSTACK(index)에 매핑을 미리 해둠
- KSTACK(i)는 어디인가

```
// kernel/memlayout.h
#define KSTACK(p) (TRAMPOLINE - ((p)+1)* 2*PGSIZE)
```

ㄷ. procinit()은 각 proc 구조체에 주소만 기록함

```
// kernel/proc.c
p->kstack = KSTACK((int) (p - proc));
```

- 여기서는 새로 할당하는 게 아니라, “너의 kernel stack 가상주소는 여기야”라고 적어둠

ㄹ. allocproc()는 그걸 실제 실행 context에 꽂음

```
// kernel/proc.c
p->context.ra = (uint64)forkret;
p->context.sp = p->kstack + PGSIZE;
```

- 여기서 context.sp가 바로 **커널 모드에서 처음 실행될 때 쓸 stack pointer**

결론

1. kclone()에서는 kernel stack을 직접 만들 필요가 없음

0x50. result

0x51. kproctest결과

```
→  project2-im0x00 git:(main) ✗ make qemu
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kern
lse -drive file=fs.img,if=none,format=raw,id=x0 -device v

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ./kproctest
kproctest: RUN clone_process_private_vm
kproctest: PASS clone_process_private_vm
kproctest: RUN clone_thread_shared_vm_and_join
kproctest: PASS clone_thread_shared_vm_and_join
kproctest: RUN join_collects_all_threads_once
kproctest: PASS join_collects_all_threads_once
kproctest: RUN wait_reaps_process_exit_status
kproctest: PASS wait_reaps_process_exit_status
kproctest: RUN wait_ignores_threads
kproctest: PASS wait_ignores_threads
kproctest: RUN join_restricted_to_leader
kproctest: PASS join_restricted_to_leader
kproctest: RUN thread_sbrk_shares_mm
kproctest: PASS thread_sbrk_shares_mm
kproctest: RUN exit_leader_drains_threads
kproctest: PASS exit_leader_drains_threads
kproctest: RUN kill_process_reports_minus_one
kproctest: PASS kill_process_reports_minus_one
kproctest: RUN kill_thread_group_by_tid
kproctest: PASS kill_thread_group_by_tid
kproctest: RUN clone_thread_limit_no_panic
kproctest: PASS clone_thread_limit_no_panic
kproctest: RUN uthread_wrappers
kproctest: PASS uthread_wrappers
kproctest: PASS all
```

0x52. clone_process_private_vm

> fork()가 내부적으로 clone(... flags=0)을 쓰더라도, process clone은 주소공간을 공유하면 안 됨

```
private_value = 111;
pid = fork();
if(pid == 0){
  private_value = 222;
  exit(WAIT_STATUS);
}
```

```
got = wait(&status);
CHECK_EQ(private_value, 111, "process clone unexpectedly shared memory");
```

- 자식이 private_value를 222로 바꿔도 부모는 여전히 111이어야 한다.
- 즉 kclone의 non-CLONE_VM 경로가 uvmcopy로 독립 주소공간을 만들었는지 본다.

0x53. clone_thread_shared_vm_and_join

> CLONE_VM thread는 주소공간을 공유하고, join()으로 tid와 stack을 회수

```
shared_value = 0;
tid = clone_one(shared_worker, &arg, &stack);
CHECK(tid > 0, "clone(CLONE_VM) failed");
join_one(tid, stack);
CHECK_EQ(shared_value, 42, "thread did not share address space");
CHECK_EQ(wait(0), -1, "wait should not reap a joined thread");
```

- thread가 바꾼 shared_value가 부모에게 보여야 하고, thread는 wait()가 아니라 join()으로만 회수되어야 함

0x54. join_collects_all_threads_once

> 여러 thread를 만들었을 때 join()이 모든 thread를 정확히 한 번씩 회수하는지 봄

```
tids[i] = clone(slot_worker, &args[i], stacks[i], 1, CLONE_VM);

int joined_tid = join(&joined_stack);
CHECK(joined_tid > 0, "join failed before all threads were collected");
CHECK(known_stack_index(joined_stack, stacks, seen, NJOIN) >= 0,
      "join returned an unknown or duplicate stack");
```

0x55. wait_reaps_process_exit_status

> wait가 child process의 pid와 exit status를 제대로 반환하는지 확인

```
pid = fork();
if(pid == 0)
  exit(WAIT_STATUS);

got = wait(&status);
CHECK_EQ(got, pid, "wait returned the wrong child pid");
CHECK_EQ(status, WAIT_STATUS, "wait returned the wrong exit status");
CHECK_EQ(wait(0), -1, "wait with no children should fail");
```

0x56. wait_ignores_threads

> kwait가 sibling thread를 child process처럼 회수하지 않는지 확인

```
tid = clone_one(shared_worker, &arg, &stack);
while(shared_value == 0)
```

```
  pause(1);

CHECK_EQ(wait(0), -1, "wait should not reap a sibling thread");
join_one(tid, stack);
```

0x57. join_restricted_to_leader

> group leader가 아닌 thread는 join()을 호출할 수 없어야 함.

```
tid = clone_one(join_from_thread_worker, 0, &stack);
join_one(tid, stack);
CHECK_EQ(join_from_thread_result, -1, "non-leader thread should not join");
```

worker 쪽 핵심:

```
join_from_thread_result = join(0);
exit(0);
```

0x58. thread_sbrk_shares_mm

> thread가 sbrk()로 늘린 heap이 같은 mm_struct를 통해 부모에게도 보여야 한다.

```
arg.oldbrk = sbrk(0);
tid = clone_one(sbrk_worker, &arg, &stack);
join_one(tid, stack);

CHECK_PTR_EQ(arg.allocated, arg.oldbrk, "thread sbrk used the wrong break");
CHECK_EQ(arg.allocated[0], 'T', "parent could not read first byte");
CHECK_PTR_EQ(sbrk(0), arg.oldbrk + PGSIZE,
             "parent did not observe shared break growth");
```

0x59. exit_leader_drains_threads

> group leader가 exit()하면 살아 있는 sibling thread들이 정리되고, 부모 wait()가 leader의 exit status를 받아야 함.

```
if(pid == 0){
  tid = clone_one(sleeper_worker, (void *)&sleeper_ready, &stack);
  while(sleeper_ready == 0)
    pause(1);

  exit(LEADER_EXIT_STATUS);
}

got = wait(&status);
CHECK_EQ(got, pid, "wait returned the wrong child pid");
CHECK_EQ(status, LEADER_EXIT_STATUS, "leader exit status was not preserved");
```

0x5A. kill_process_reports_minus_one

> 일반 child process를 kill()하면 wait() status가 -1이어야 함.

```
CHECK_EQ(kill(pid), 0, "kill returned failure for live child");
got = wait(&status);
CHECK_EQ(got, pid, "wait returned the wrong killed child pid");
CHECK_EQ(status, -1, "killed child should report status -1");
CHECK_EQ(kill(pid), -1, "kill should fail after child is reaped");
```

0x5B. kill_thread_group_by_tid

> leader pid가 아니라 thread tid로 kill()해도 thread group 전체가 죽어야 함.

```
tid = clone_one(sleeper_worker, (void *)&sleeper_ready, &stack);
CHECK_EQ(write(fds[1], &tid, sizeof(tid)), sizeof(tid),
         "child failed to report thread tid");
```

- 부모쪽

```
CHECK_EQ(kill(tid), 0, "kill by thread tid failed");
got = wait(&status);
CHECK_EQ(got, pid, "wait should reap the killed group leader");
CHECK_EQ(status, -1, "killed thread group should report status -1");
```

0x5C. uthread_wrappers

> user-level helper인 thread_create, thread_join이 stack malloc/free 생명주기를 제대로 감싸는지 보기.

```
tid = thread_create(wrapper_worker, &arg, 1);
CHECK(tid > 0, "thread_create failed");
joined_tid = thread_join();
CHECK_EQ(joined_tid, tid, "thread_join returned the wrong tid");
CHECK_EQ(wrapper_value, 777, "wrapper thread did not share memory");
```